

UML Class Diagram

1. What is UML?

- UML is a **visual modeling language** for software design.
 - It shows **structure** (classes, interfaces, objects) and **behavior** (methods, interactions, sequences).
 - It's **not a programming language**, but it maps closely to Java.
-

2. Key UML Diagram Types for Java Developers

A. Class Diagram

- **Purpose:** Shows classes, their attributes, methods, and relationships.
- **Java Mapping:**
 - Class → `class MyClass { ... }`
 - Interface → `interface MyInterface { ... }`
 - Attribute → `private int age;`
 - Method → `public void doSomething() {...}`

Notation Example:

```

-----
|      Person      |
|-----|
| - name: String   |
| - age: int       |
|-----|
| + getName(): String|
| + setAge(a: int)  |
|-----|

```

- **Symbols:**

- + → public
- - → private
- # → protected

B. Interface and Abstract Class

- Abstract classes: Italicize class name or use {**abstract**}.
- Interfaces: Use <<**interface**>> stereotype.

```

-----
<<interface>>
|  Flyable      |
|-----|
| + fly(): void  |
|-----|

```

C. Relationships

- **Association:** Solid line (some relationship between classes) {eg, Student to Sport, 0..* 0..*}
- **Aggregation:** Hollow diamond (weak has-a relationship, part of, both can exist independently, empty diamond denotes the whole object) {eg, Airliner to Crew, 0..* 0..*}
- **Composition:** Filled diamond (stronger has-a relationship, part-of, contained parts are exclusive to the whole, filled diamond denotes the whole object, instantiated at creation) {eg, House to Room, _ 1..*}
- **Inheritance:** Solid line with a triangle pointing to the parent (the standard way is the arrow pointing upward, i.e., superclasses are always toward the top, no need to put any of the inherited superclasses' attributes and behaviors into the subclass. The superclasses are the generalized classes, and the subclasses are the specialized classes.)
- **Implementation:** Dashed line with triangle (interface implemented, surrounded with <<interface>> on top of class name, interface at the top end of the arrow, and class at the bottom, should duplicate interface behaviors in the implementation class)

Java Example:

```
class Engine { ... }
class Car {
    private Engine engine; // Aggregation
}

class Vehicle { ... }
class Bike extends Vehicle { ... } // Inheritance
```

UML Representation:

```
Car <----- Engine    // Aggregation
Bike ---|> Vehicle    // Inheritance
```


D. Object Diagram

- Shows **actual objects** and their **values** at runtime.
- Less common for static Java design, but useful for testing/debugging.

```
-----  
| person: Person    |  
-----  
| name = "John"     |  
| age = 30           |  
-----
```

E. Sequence Diagram (Behavior)

- Shows **object interaction over time**.
- Helpful for mapping method calls.

```
Person          Bank  
|               |  
|  withdraw()  |  
|----->|  
|               |  
|  success()   |  
|<-----|
```

F. Package Diagram

- Groups related classes.
- Java analogy: Packages (`com.myapp.service`).

3. Tips for Java Programmers

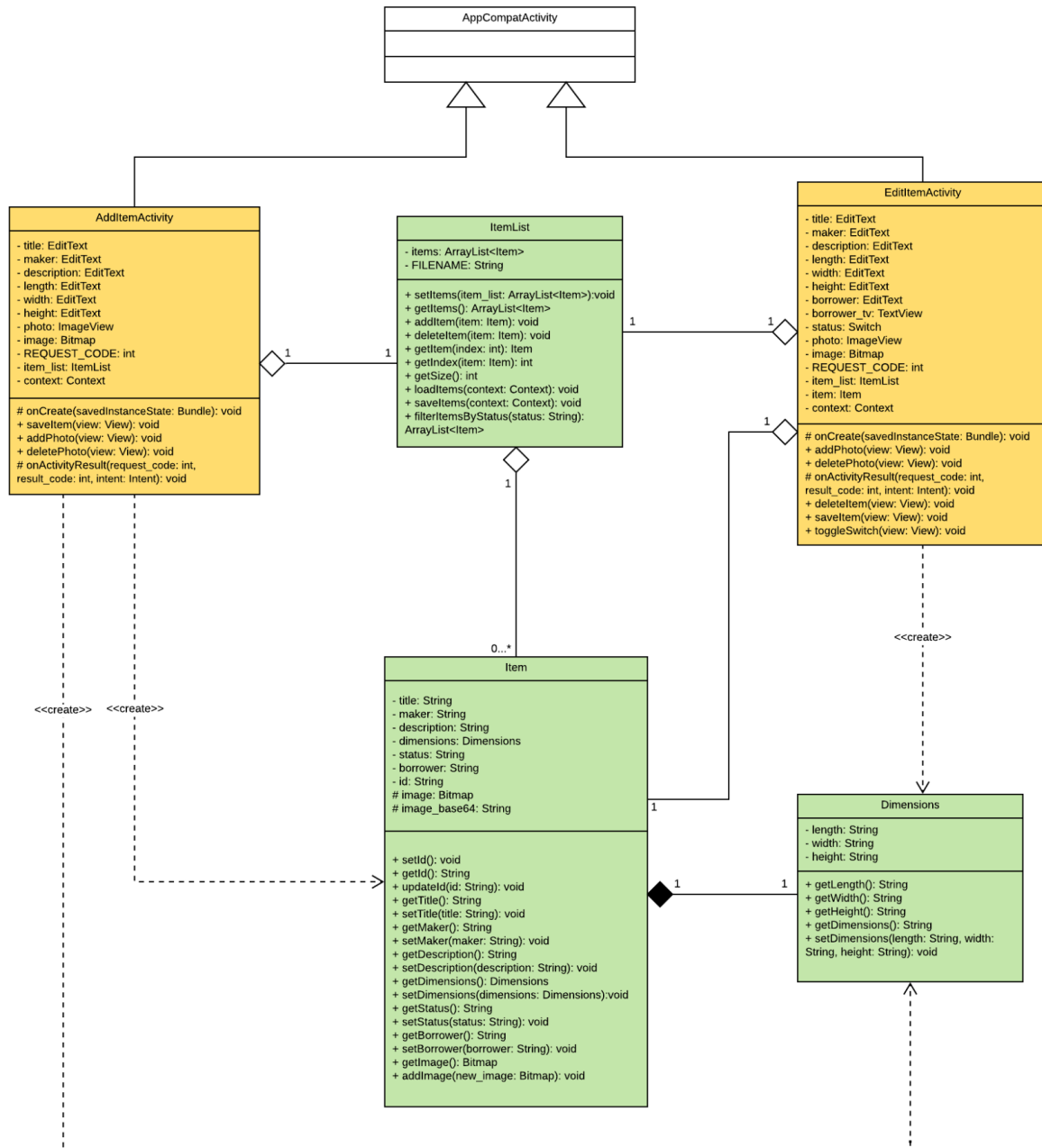
1. **Start with class diagrams** — map your Java classes and relationships first.
 2. Use **interfaces and abstract classes** to show polymorphism.
 3. **Method signatures** in UML can include parameters and return types.
 4. Keep diagrams **simple** for understanding, complex diagrams can be broken into multiple diagrams.
-

User Items Drawing

**UML Class Diagram for Application
(Items Only)
Solution**

SDA 1.1 UML Class Diagram Solution

Software Design and Architecture | November 25, 2019



UML Sequence Diagram

What is a Sequence Diagram?

- A **Sequence Diagram** is a type of **UML Interaction Diagram** that shows **how objects interact to complete a specific task**.
- It emphasizes the **time order of messages** exchanged between different objects/components.
- Think of it as a **"time-based storyboard"** for your system.

👉 In short: **It answers "Who does what, and in what order?"**

Key Elements of a Sequence Diagram

1. Objects/Actors (Lifelines)

- Represented by rectangles at the top, with a dashed vertical line below (the **lifeline**).
- Examples: **User**, **Controller**, **Service**, **Database**.

2. Activation Box (Execution Occurrence)

- Narrow rectangle on a lifeline.
- Shows when an object is actively performing an action (e.g., executing a method).

3. Messages (Arrows)

- Show communication between lifelines.
- Types:
 - **Synchronous message** (solid line with filled arrowhead) → caller waits for response.

- **Asynchronous message** (solid line with open arrowhead) → caller doesn't wait.
- **Return message** (dashed line with arrowhead) → response/result.

4. Actors

- External entities (e.g., User, external system). Shown as stick figures or named rectangles.

5. Lifeline Destruction (X)

- An **X** at the end of a lifeline → object is destroyed at that point.

Example: User Login Flow

Imagine a user logs in to an application.

Steps:

1. User enters credentials.
2. UI sends request to **AuthController**.
3. **AuthController** calls **AuthService**.
4. **AuthService** queries **UserDatabase**.
5. Database returns user details.
6. **AuthService** validates and returns result.
7. **AuthController** shows success/failure.

Diagram Flow (conceptually):

```
User → AuthController : enterCredentials()
AuthController → AuthService : validateUser()
AuthService → UserDatabase : fetchUser()
UserDatabase → AuthService : return userDetails
```

```
AuthService → AuthController : return authResult  
AuthController → User : showResult()
```

✓ Why Use Sequence Diagrams?

- Clarify **system behavior** before coding.
 - Useful for **designing interactions** in distributed systems (e.g., microservices, APIs).
 - Helps in **identifying responsibilities** of each component.
 - Easy to translate into **unit tests** or **integration flow checks**.
-

vs Sequence Diagram vs Other UML Diagrams

- **Class Diagram** → *static* structure (what objects exist, their relationships).
 - **Sequence Diagram** → *dynamic* behavior (how objects interact over time).
 - **Activity Diagram** → flow of activities without focusing on object interaction.
-



Scenario: User Login

- A **User** tries to log in.
 - Credentials go to the **LoginController**.
 - The controller asks the **AuthService** to validate.
 - The service queries the **UserDatabase**.
 - The database returns details, validation happens, and result is shown.
-

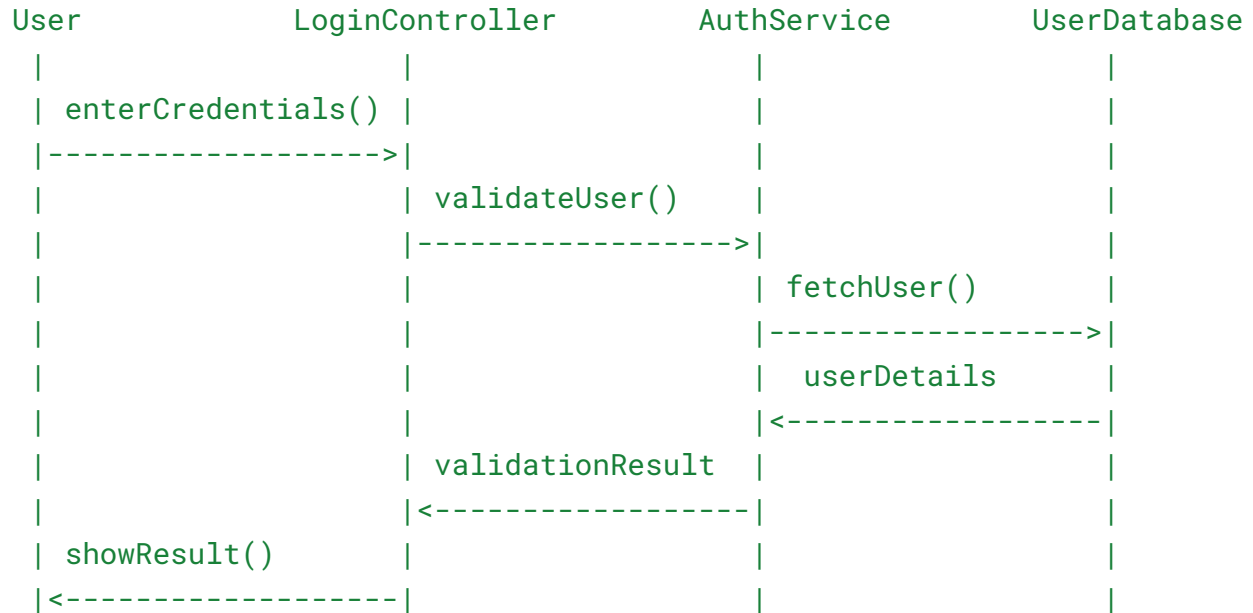
◆ Sequence Diagram (PlantUML Example)

You can paste this into any PlantUML editor to visualize:

```
@startuml
actor User
participant LoginController
participant AuthService
database UserDatabase

User -> LoginController : enterCredentials(username, password)
LoginController -> AuthService : validateUser(username, password)
AuthService -> UserDatabase : fetchUser(username)
UserDatabase --> AuthService : userDetails
AuthService --> LoginController : validationResult
LoginController --> User : showResult(success/failure)
@enduml
```

◆ How It Looks Conceptually (ASCII)



Key Points in This Example

- **Actor** (**User**) starts the interaction.
 - **Messages** are synchronous (solid arrows).
 - **Return messages** are dashed arrows (in PlantUML automatically shown).
 - **Database** is shown as a special **database** stereotype.
-

Notes

When creating sequence diagrams, first, you **use a box to represent the role played by an object**. The role is typically **labeled by the name of the class** for the object. Second, you **use vertical dotted lines**, known as **lifelines**, **to represent an object as time passes by**. Finally, you **use arrows to show messages** that are sent from one object to another.

In a sequence diagram, if **one object sends a message to another object** or objects, we **denote this by drawing a solid line arrow** from the sender to the receiver. To **return data** and to control back to initiating objects, we would use a **dotted line arrow**.

When an **object is activated**, we denote this on our sequence diagram using **small rectangles on the objects lifeline**.

UML State Diagram

Here's a crisp, practical guide to **UML State (State Machine) Diagrams**, with flight-ticket booking examples you can reuse.

What a State Diagram models

A **State Machine diagram** shows the **lifecycle of a single object (or feature)**: the discrete **states** it can be in, **events** that trigger **transitions**, and **actions** that run along the way. Use it when behavior **depends on history** (protocols, order, timeouts, retries).

Good candidates: **Booking**, **Payment**, **SeatHold**, **Gate**, **Baggage**, **BoardingPass**.

Core building blocks

- **State**: a condition during the life of an object (e.g., **PendingPayment**, **Confirmed**).
 - Internal behaviors:
 - **entry** / ... runs when entering the state
 - **do** / ... runs while in the state (longer activity)
 - **exit** / ... runs when leaving the state
 - **State invariant**: condition that must always hold while in the state.
- **Transition**: arrow from one state to another. Label = **trigger** [**guard**] / **effect**
 - **Trigger**: event that fires it (signal, call, **time**, **change**)
 - **Guard**: boolean that must be true
 - **Effect**: quick action that executes with the transition
- **Event types**
 - **Signal/Call**: **pay()**, **cancel()**
 - **Time**: **after(10m)**, **at(2025-08-26T18:00)**

- **Change:** `when(retryCount >= 3)`
- **Pseudo-states** (control nodes)
 - **Initial** `[*]` → first state
 - **Final** `[*]` → lifecycle end (for that machine)
 - **Choice** `<<choice>>` → evaluate guards at runtime
 - **Junction** `<<junction>>` → merge/split without guard evaluation
 - **Fork / Join** `<<fork>>` / `<<join>>` → parallel regions begin/end
 - **History:** **shallow** `H` and **deep** `H*` → re-enter a composite state where you left off
 - **Entry/Exit points** → controlled entry/exit of composite states
 - **Terminate** → kill the machine (rare; stronger than Final)
- **Composite states & Regions**
 - A state can **nest substates** (hierarchy).
 - A composite state can have **orthogonal regions** (true concurrency).

Reading labels—by example

`authorize [cardValid] / log("authorized")`

- Trigger = `authorize`
- Guard = `cardValid`
- Effect = `log(...)`

`after(10m) / releaseSeats()`

- Time event, fires 10 minutes after entering the state, then effect runs.

Example 1 — Booking lifecycle (airline website)

This models a **single Booking** object from selection to ticket issuance, including timeouts and payment failure.

```
@startuml
hide empty description
skinparam state {
    BackgroundColor<<busy>> #EEE
    BorderColor #888
}

[*] --> Selecting : createBooking / startTimer(hold=10m)

state Selecting <<busy>> {
    [*] --> CollectingDetails
    CollectingDetails --> SeatHeld : passengerInfoEntered / holdSeats()
    SeatHeld --> CollectingDetails : editPassenger()
}

Selecting --> PendingPayment : checkout
Selecting --> Cancelled : cancel

PendingPayment : entry / startTimer(payment=10m)
PendingPayment : do / showPaymentForm()
PendingPayment --> PaymentProcessing : pay(paymentInfo)
PendingPayment --> Expired : after(10m) / releaseSeats()

state PaymentProcessing {
    [*] --> Authorizing
    Authorizing --> Authorized : authOk
    Authorizing --> PaymentFailed : authFail / recordFailure()
    Authorized --> Capturing : capture
```

```

    Capturing --> Captured : captureOk
    Capturing --> PaymentFailed : captureFail / refundIfNeeded()
}

PaymentProcessing --> Confirmed : Captured / issueETicket()
PaymentProcessing --> PendingPayment : PaymentFailed [retryLeft] /
showRetry()
PaymentProcessing --> Cancelled : PaymentFailed [!retryLeft] /
releaseSeats()

Confirmed : entry / setEmail(), showConfirmation()
Cancelled : entry / notifyCancel(), releaseSeats()
Expired : entry / notifyExpiry(), releaseSeats()

Confirmed --> [*]
Cancelled --> [*]
Expired --> [*]
@enduml

```

What this shows

- **Timeouts** with `after(10m)` to expire a hold
- **Hierarchical** `PaymentProcessing` with substates (`Authorizing`, `Capturing`)
- **Retry path** using a **guard** `[retryLeft]`
- Side effects clearly placed on `entry` and transitions

Example 2 — Parallel checkout (orthogonal regions)

Suppose during checkout you **collect passenger details** and **upsell ancillaries** in parallel; both must complete before payment.

```
@startuml
```

```

[*] --> Checkout

state Checkout {
  [*] --> Parallel

  state Parallel {
    --
    state PassengerDetails {
      [*] --> Missing
      Missing --> Completed : detailsEntered
    }
    --
    state Ancillaries {
      [*] --> Skipped
      Skipped --> Selected : addBagsOrSeats
      Selected --> Completed : confirmAncillaries
    }
    --
    [*] --> PassengerDetails
    [*] --> Ancillaries

    CompletedAll <<join>>
    PassengerDetails.Completed --> CompletedAll
    Ancillaries.Completed --> CompletedAll
  }

  Parallel.CompletedAll --> Payment
}

Payment --> [*] : pay
@enduml

```

Why this matters: demonstrates **orthogonal regions** with a **join**—true concurrency, not just sequential steps.

Advanced elements you'll actually use

- **History**: if the user backs out of `Checkout` and returns, use **history** to resume where they left.
- **Deferred events**: states can **defer** events they can't handle yet (e.g., defer `refund` until `Captured`).
- **State invariants**: annotate rules like “seat hold must remain valid” in `PendingPayment`.

Common modeling pitfalls (and fixes)

- **Mixing workflow with lifecycle**
State machines are best for **one object's lifecycle** (e.g., `Booking`). Use **Activity Diagrams** for organization-wide process flows.
- **Overusing actions on transitions**
Prefer meaningful **entry/exit** logic for durability and testability.
- **Ambiguous guards**
Keep guards **pure** (no side effects) and **mutually exclusive** on a `choice`.
- **Ignoring time**
Ticketing needs **timeouts** (holds, payments). Model them explicitly with `after(...)`.
- **Fake concurrency**
If tasks truly run in parallel (UI + services), use **orthogonal regions** and **fork/join**.

From diagram to code (Java-flavored)

You can implement with the **State pattern**, or a framework like **Spring Statemachine**.

Events

```
enum BookingEvent { CHECKOUT, PAY, CANCEL, AUTH_OK, AUTH_FAIL,
CAPTURE, CAPTURE_OK, CAPTURE_FAIL, EXPIRE }
```

States

```
enum BookingState { SELECTING, PENDING_PAYMENT, PAYMENT_PROCESSING,
CONFIRMED, CANCELLED, EXPIRED }
```

Guard example

```
boolean retryLeft(Booking b) { return b.getRetries() < MAX_RETRIES; }
```

Timed transitions

- Use a scheduler to fire **EXPIRE** after 10 minutes in **PENDING_PAYMENT**.
- Persist current state so restarts resume correctly (consider **history** semantics if you re-enter a composite state).

When to choose State Machines

- Protocols (payment, check-in, boarding)
- Long-lived processes with **timeouts** and **retries**
- Complex **error handling** or **compensation** (e.g., refund if capture fails)
- Systems that must be **auditable** (every transition logged)

Quick checklist for high-quality state models

- Clear **single owner** (one object per machine)
- Explicit **initial** and **final** (or **terminate**) points
- Time and change events modeled
- Guards are deterministic, side-effect free

- Effects are idempotent (safe to retry)
 - Parallelism modeled with orthogonal regions if truly concurrent
 - Tests map 1:1 to transitions (happy, error, timeout, cancel)
-

Notes

State diagrams can describe a single object and illustrate how that object behaves in response to a series of events in your system.

A state is the way an object exists at a particular point in time. The state of an object is determined by the values of its attributes.

States are drawn as rounded rectangles. Let me explain states in more detail, each state has three important sections: a state name, state variables, and activities. Each state should at least have a state name. A state name is, as it sounds, the name of the state, these names should be meaningful for the states of your object.

For example, a car in reverse would have a state named reverse. Or a vending machine in a waiting, or idle state, would have a state named idle.

State variables are data relevant to the state of the object.

For example, using a course as an object, a relevant variable is the number of students enrolled. The course would be in state full, if this variable was at the course capacity.

Activities are actions that are performed when in a certain state, and they're displayed at the bottom. There are three types of activities for each state: entry, exit, and do. Entry activities are actions that occur when the state is just entered from another state. Exit activities are actions that occur when the state is exited and moves on to another state. And do activities are actions that occur once, or multiple times, while the object is in a certain state.

To explain these activities, let's use a traditional alarm clock that uses a bell as an alarm. An entry activity for when a clock enters the ringing state is that the clock releases a spring to ring a bell. When the clock leaves the ringing state, it will relock the spring as an exit activity. When the clock is actually in its ringing state, the clock continuously rings the bell, and this is its do activity.

Events that could change a state label transitions between the states. You draw these transitions with arrows from one state to another. Each transition arrow will always have an event, and may have a guard condition and an action. The transition and action happens from a given state if the event occurs and the condition is true.

Termination represents an object being destroyed or the process being completed, and is drawn as a circle with a filled circle inside.

Model Checking

Model checking is a systematic check of your system's state model in all its possible states. Basically, you check all the various states of your software and find that there are any errors by simulating different events that would change the states and variables of your software. Model checking is a technique that does state exploration. It says, given that you start in this current state, explore all the possible ways you can go into the future and reason about what all those possible future states can look like.

Model checkers generate a State Model from your code. A state model is an abstract state machine that can be in one of various states. The model checker then checks that the state model conforms to certain behavioral properties.

There are three different phases to performing model checking: the modeling phase, the running phase, and the analysis phase.

The modeling phase comes first. This is where you enter the model description. During the modeling phase, you can perform some Sanity checks.

The second phase is the running phase, this is when you run the model checker to see how your model conforms to the desired properties that you've wrote in the modeling phase.

The last phase is the analysis phase. This is where you check if all the desired properties are satisfied and if any are violated.

Violations are called counterexamples. Your model checker will provide descriptions of violations in your system so that you can analyze how it got to that state.

Using the information from the model checker, you advice yourself where to fix any problems, then run through the phases again. You do this until you can be sure that your software is correct with respect to the desired properties.